

INSTITUTO TECNOLÓGICO METROPOLITANO — ITM

Aplicaciones y Servicios Web | Código: 580202009

GUÍA DE TRABAJO No. 4 — TALLER EVALUATIVO 20%

Implementación de un CRUD en Python con Pruebas Unitarias

Estudiante(s):	YIMMY ALEXANDER DURANGO ECHAVARRIA
Fecha de entrega:	24 de May de 2026
Docente:	Jonathan Sánchez Giraldo

1. Introducción

El presente documento describe el desarrollo de un sistema CRUD (Create, Read, Update, Delete) para la gestión de un registro de productos, implementado en Python. El proyecto cumple con los requisitos de la Guía de Trabajo No. 4 de la asignatura Aplicaciones y Servicios Web: lenguaje Python, almacenamiento en memoria con persistencia en JSON, pruebas unitarias por cada operación y publicación del código en GitHub con análisis de calidad en SonarCloud.

2. Configuración Inicial y Estructura del Proyecto

El proyecto contiene los siguientes archivos:

Archivo	Descripción
<code>productos.py</code>	Clase ProductoCRUD: lógica de negocio CRUD + persistencia JSON.
<code>main.py</code>	Menú interactivo en consola. Guarda datos en <code>productos.json</code> .
<code>test_productos.py</code>	22 pruebas unitarias (unittest) para cada operación CRUD.

2.1 Captura — Estructura del proyecto en VS Code

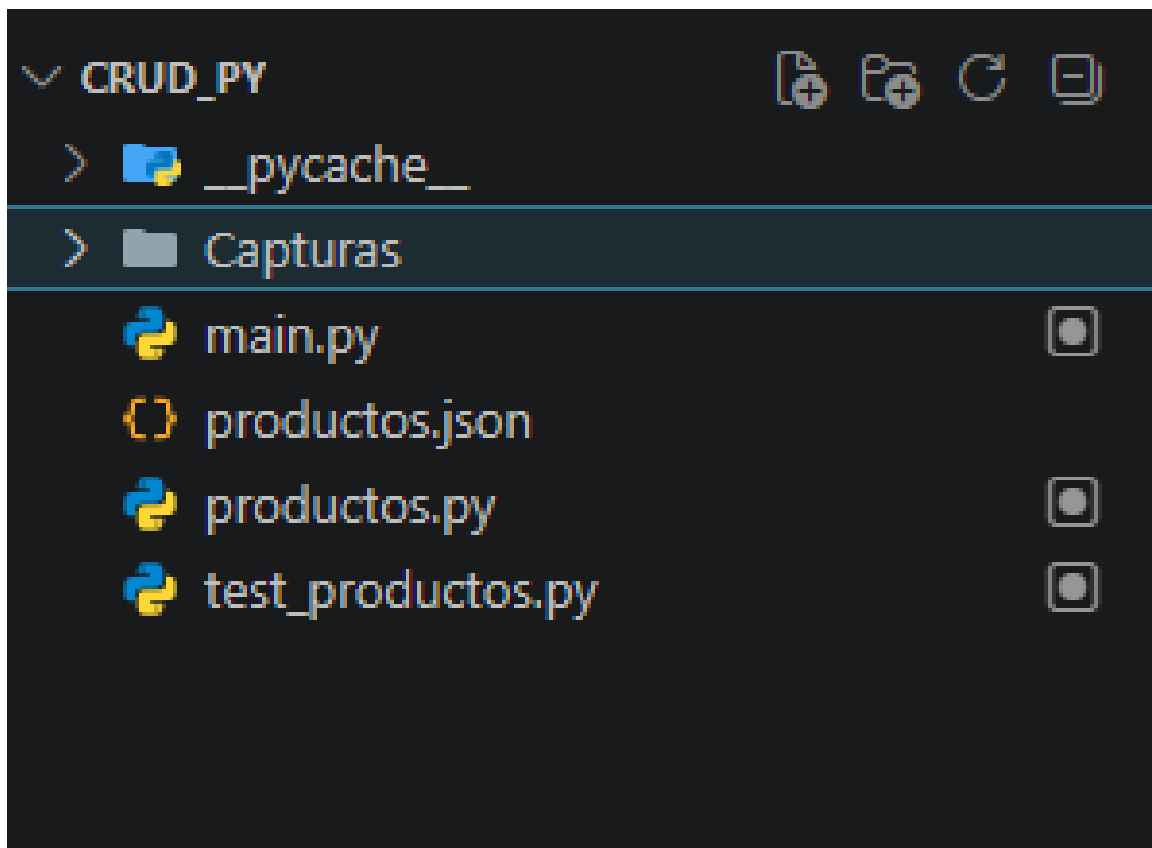


Figura: Estructura del proyecto en VS Code Explorer

3. Modelo de Datos — Producto

Campo	Tipo	Descripción
id	int	Identificador único del producto.
nombre	str	Nombre comercial del producto.
descripcion	str	Breve descripción del producto.
precio	float	Precio unitario (≥ 0).
cantidad	int	Unidades disponibles en inventario (≥ 0).

4. Implementación del CRUD

4.1 Código fuente — productos.py

Clase ProductoCRUD con todas las operaciones y persistencia en JSON:

```
import json, os

class ProductoCRUD:
    def __init__(self, archivo: str = None):
        self._productos: dict = {}
        self._archivo = archivo
        if archivo and os.path.exists(archivo):
            self._cargar()

    # CREATE
    def crear(self, id, nombre, descripcion, precio, cantidad):
        if id in self._productos:
            raise ValueError(f"Ya existe un producto con id={id}")
        if precio < 0: raise ValueError("El precio no puede ser negativo")
        if cantidad < 0: raise ValueError("La cantidad no puede ser negativa")
        producto = {"id": id, "nombre": nombre, "descripcion": descripcion,
                    "precio": precio, "cantidad": cantidad}
        self._productos[id] = producto
        self._guardar()
        return dict(producto)

    # READ
    def leer(self, id):
        if id not in self._productos:
            raise KeyError(f"No existe producto con id={id}")
        return dict(self._productos[id])

    def leer_todos(self):
        return [dict(p) for p in self._productos.values()]

    # UPDATE
    def actualizar(self, id, **campos):
```

```

        if id not in self._productos:
            raise KeyError(f"No existe producto con id={id}")
        campos_validos = {"nombre", "descripcion", "precio", "cantidad"}
        invalidos = set(campos) - campos_validos
        if invalidos: raise ValueError(f"Campos no válidos: {invalidos}")
        if "precio" in campos and campos["precio"] < 0:
            raise ValueError("El precio no puede ser negativo")
        if "cantidad" in campos and campos["cantidad"] < 0:
            raise ValueError("La cantidad no puede ser negativa")
        self._productos[id].update(campos)
        self._guardar()
        return dict(self._productos[id])

# DELETE
def eliminar(self, id):
    if id not in self._productos:
        raise KeyError(f"No existe producto con id={id}")
    eliminado = self._productos.pop(id)
    self._guardar()
    return dict(eliminado)

def _guardar(self):
    if self._archivo:
        with open(self._archivo, "w", encoding="utf-8") as f:
            json.dump(list(self._productos.values()), f, ensure_ascii=False, indent=2)

def _cargar(self):
    with open(self._archivo, "r", encoding="utf-8") as f:
        datos = json.load(f)
    self._productos = {p["id"]: p for p in datos}

```

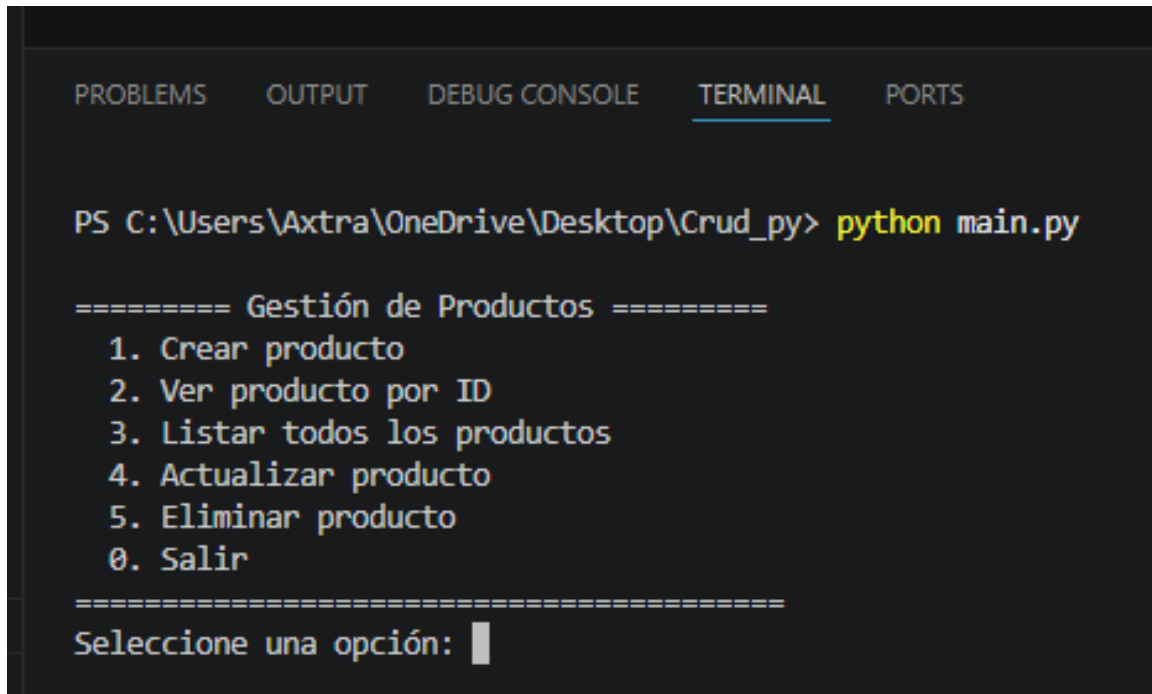
4.2 Descripción de operaciones

- **crear:** Valida id único, precio ≥ 0 y cantidad ≥ 0 . Lanza ValueError si falla.
- **leer:** Retorna copia del producto por id. Lanza KeyError si no existe.
- **leer_todos:** Retorna lista completa de productos. Lista vacía si no hay ninguno.
- **actualizar:** Modifica campos permitidos. Valida existencia y valores numéricos.
- **eliminar:** Extrae el producto del diccionario y lo devuelve. Lanza KeyError si no existe.

5. Evidencia de Ejecución — Operaciones CRUD

5.1 Menú principal

Ejecución de python main.py mostrando el menú interactivo.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

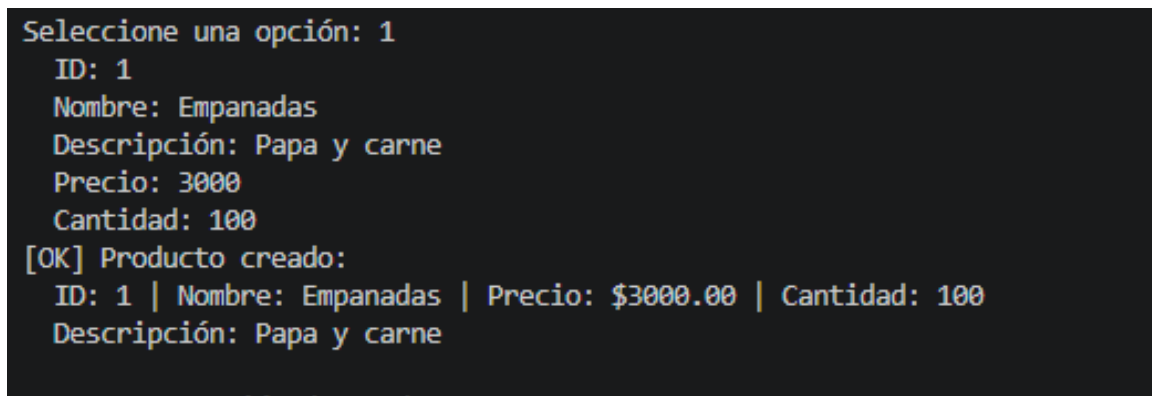
PS C:\Users\Axta\OneDrive\Desktop\Crud_py> python main.py

===== Gestión de Productos =====
  1. Crear producto
  2. Ver producto por ID
  3. Listar todos los productos
  4. Actualizar producto
  5. Eliminar producto
  0. Salir
=====
Seleccione una opción: █
```

Figura: Menú principal — python main.py

5.2 Crear producto — caso exitoso

Se ingresa un producto nuevo con todos sus campos. El sistema responde con [OK].



```
Seleccione una opción: 1
ID: 1
Nombre: Empanadas
Descripción: Papa y carne
Precio: 3000
Cantidad: 100
[OK] Producto creado:
ID: 1 | Nombre: Empanadas | Precio: $3000.00 | Cantidad: 100
Descripción: Papa y carne
```

Figura: Crear producto exitoso

5.3 Crear producto — caso de error (ID duplicado)

Se intenta crear un producto con un ID ya registrado. El sistema responde con [ERROR].

```
Seleccione una opción: 1
  ID: 1
  Nombre: Coca-cola
  Descripción: 400ml
  Precio: 5000
  Cantidad: 10
[OK] Producto creado:
  ID: 1 | Nombre: Coca-cola | Precio: $5000.00 | Cantidad: 10
  Descripción: 400ml

===== Gestión de Productos =====
  1. Crear producto
  2. Ver producto por ID
  3. Listar todos los productos
  4. Actualizar producto
  5. Eliminar producto
  0. Salir
=====
Seleccione una opción: 1
  ID: 1
  Nombre: Pizza
  Descripción: Familiar
  Precio: 5000
  Cantidad: 1
[ERROR] Ya existe un producto con id=1

===== Gestión de Productos =====
  1. Crear producto
  2. Ver producto por ID
  3. Listar todos los productos
  4. Actualizar producto
  5. Eliminar producto
  0. Salir
```

Figura: Crear producto — ID duplicado

5.4 Leer producto por ID — caso exitoso

Se consulta un producto existente por su ID y se muestran sus datos.

```
=====
Seleccione una opción: 3
  ID: 1 | Nombre: Empanadas | Precio: $3000.00 | Cantidad: 100
  Descripción: Papa y carne

===== Gestión de Productos =====
  1. Crear producto
  2. Ver producto por ID
  3. Listar todos los productos
  4. Actualizar producto
  5. Eliminar producto
  0. Salir
```

Figura: Leer producto por ID exitoso

5.5 Listar todos los productos

Se listan todos los productos actualmente registrados en el sistema.

```
Nueva cantidad: 10
[OK] Producto actualizado:
  ID: 1 | Nombre: CocaCola | Precio: $3500.00 | Cantidad: 10
  Descripción: Zero 300ml

===== Gestión de Productos =====
  1. Crear producto
  2. Ver producto por ID
  3. Listar todos los productos
  4. Actualizar producto
  5. Eliminar producto
  0. Salir

=====
Seleccione una opción: 3
  ID: 1 | Nombre: CocaCola | Precio: $3500.00 | Cantidad: 10
  Descripción: Zero 300ml
```

Figura: Listar todos los productos

5.6 Actualizar producto — caso exitoso

Se modifican uno o más campos de un producto existente. El sistema confirma con [OK].

```
Seleccione una opción: 4
  ID del producto a actualizar: 1
  (Deje en blanco los campos que no desee cambiar)
  Nuevo nombre: Cocacola
  Nueva descripción: Zero 300ml
  Nuevo precio: 3500
  Nueva cantidad: 10
[OK] Producto actualizado:
  ID: 1 | Nombre: Cocacola | Precio: $3500.00 | Cantidad: 10
  Descripción: Zero 300ml

===== Gestión de Productos =====
  1. Crear producto
  2. Ver producto por ID
  3. Listar todos los productos
  4. Actualizar producto
  5. Eliminar producto
  0. Salir
```

Figura: Actualizar producto exitoso

5.7 Actualizar producto — caso de error (ID no existe)

Se intenta actualizar un producto con ID inexistente. El sistema responde con [ERROR].

```
=====
Seleccione una opción: 5
  ID del producto a eliminar: 1
[OK] Producto eliminado: Cocacola (ID=1)

===== Gestión de Productos =====
  1. Crear producto
  2. Ver producto por ID
  3. Listar todos los productos
  4. Actualizar producto
  5. Eliminar producto
  0. Salir

=====
Seleccione una opción: 3
No hay productos registrados.
```

Figura: Actualizar producto — ID inexistente

5.8 Eliminar producto — caso exitoso

Se elimina un producto existente. El sistema confirma con [OK] Producto eliminado.

```
===== Gestión de Productos =====
1. Crear producto
2. Ver producto por ID
3. Listar todos los productos
4. Actualizar producto
5. Eliminar producto
0. Salir
=====
Seleccione una opción: 5
  ID del producto a eliminar: 1
[OK] Producto eliminado: CocaCola (ID=1)
```

Figura: Eliminar producto exitoso

5.9 Eliminar producto — caso de error (ID no existe)

Se intenta eliminar un producto con ID inexistente. El sistema responde con [ERROR].

```
===== Gestión de Productos =====
1. Crear producto
2. Ver producto por ID
3. Listar todos los productos
4. Actualizar producto
5. Eliminar producto
0. Salir
=====
Seleccione una opción: 5
  ID del producto a eliminar: 500
[ERROR] 'No existe producto con id=500'
```

Figura: Eliminar producto — ID inexistente

6. Pruebas Unitarias

Se implementaron 22 pruebas unitarias con el módulo unittest de Python, agrupadas en cuatro clases (una por operación CRUD). Cada clase tiene al menos un caso exitoso y uno de error.

Operación	Exitosas	Error	Total
Crear	2	3	5
Leer	3	2	5
Actualizar	3	4	7
Eliminar	3	2	5

6.1 Código fuente — test_productos.py

```
import unittest
from productos import ProductoCRUD

class TestCrear(unittest.TestCase):
    def setUp(self): self.crud = ProductoCRUD()

    def test_crear_producto_retorna_datos_correctos(self):
        p = self.crud.crear(1, "Laptop", "Computadora portátil", 2500.0, 10)
        self.assertEqual(p["id"], 1); self.assertEqual(p["precio"], 2500.0)

    def test_crear_producto_queda_almacenado(self):
        self.crud.crear(2, "Mouse", "Periférico", 50.0, 25)
        self.assertEqual(self.crud.leer(2)["nombre"], "Mouse")

    def test_crear_id_duplicado_lanza_ValueError(self):
        self.crud.crear(1, "Laptop", "Desc", 2500.0, 10)
        with self.assertRaises(ValueError): self.crud.crear(1, "Mouse", "Desc2", 50.0, 5)

    def test_crear_precio_negativo_lanza_ValueError(self):
        with self.assertRaises(ValueError): self.crud.crear(3, "Teclado", "Desc", -100.0, 5)

    def test_crear_cantidad_negativa_lanza_ValueError(self):
        with self.assertRaises(ValueError): self.crud.crear(4, "Monitor", "Desc", 500.0, -1)

class TestLeer(unittest.TestCase):
    def setUp(self):
        self.crud = ProductoCRUD()
        self.crud.crear(1, "Laptop", "Computadora portátil", 2500.0, 10)
        self.crud.crear(2, "Mouse", "Periférico", 50.0, 25)

    def test_leer_producto_existente(self):
        self.assertEqual(self.crud.leer(1)["nombre"], "Laptop")

    def test_leer_todos_devuelve_lista_completa(self):
        self.assertEqual(len(self.crud.leer_todos()), 2)

    def test_leer_todos_vacio_devuelve_lista_vacia(self):
        self.assertEqual(ProductoCRUD().leer_todos(), [])
```

```

def test_leer_id_inexistente_lanza_KeyError(self):
    with self.assertRaises(KeyError): self.crud.leer(999)

def test_leer_devuelve_copia_independiente(self):
    p = self.crud.leer(1); p["nombre"] = "Cambiado"
    self.assertEqual(self.crud.leer(1) ["nombre"], "Laptop")

class TestActualizar(unittest.TestCase):
    def setUp(self):
        self.crud = ProductoCRUD()
        self.crud.crear(1, "Laptop", "Computadora portátil", 2500.0, 10)

    def test_actualizar_precio_y_cantidad(self):
        p = self.crud.actualizar(1, precio=2000.0, cantidad=8)
        self.assertEqual(p["precio"], 2000.0); self.assertEqual(p["cantidad"], 8)

    def test_actualizar_nombre(self):
        self.assertEqual(self.crud.actualizar(1, nombre="Laptop Pro") ["nombre"], "Laptop
Pro")

    def test_actualizar_descripcion(self):
        self.assertEqual(self.crud.actualizar(1, descripcion="Nueva") ["descripcion"],
"Nueva")

    def test_actualizar_producto_inexistente_lanza_KeyError(self):
        with self.assertRaises(KeyError): self.crud.actualizar(999, nombre="X")

    def test_actualizar_precio_negativo_lanza_ValueError(self):
        with self.assertRaises(ValueError): self.crud.actualizar(1, precio=-50.0)

    def test_actualizar_cantidad_negativa_lanza_ValueError(self):
        with self.assertRaises(ValueError): self.crud.actualizar(1, cantidad=-5)

    def test_actualizar_campo_invalido_lanza_ValueError(self):
        with self.assertRaises(ValueError): self.crud.actualizar(1, color="rojo")

class TestEliminar(unittest.TestCase):
    def setUp(self):
        self.crud = ProductoCRUD()
        self.crud.crear(1, "Laptop", "Computadora portátil", 2500.0, 10)
        self.crud.crear(2, "Mouse", "Periférico", 50.0, 25)

    def test_eliminar_producto_existente(self):
        self.assertEqual(self.crud.eliminar(1) ["id"], 1)
        self.assertEqual(len(self.crud.leer_todos()), 1)

    def test_eliminar_producto_ya_no_accesible(self):
        self.crud.eliminar(1)
        with self.assertRaises(KeyError): self.crud.leer(1)

    def test_eliminar_reduce_conteo(self):

```

```

        self.crud.eliminar(1); self.assertEqual(len(self.crud.leer_todos()), 1)

    def test_eliminar_producto_inexistente_lanza_KeyError(self):
        with self.assertRaises(KeyError): self.crud.eliminar(999)

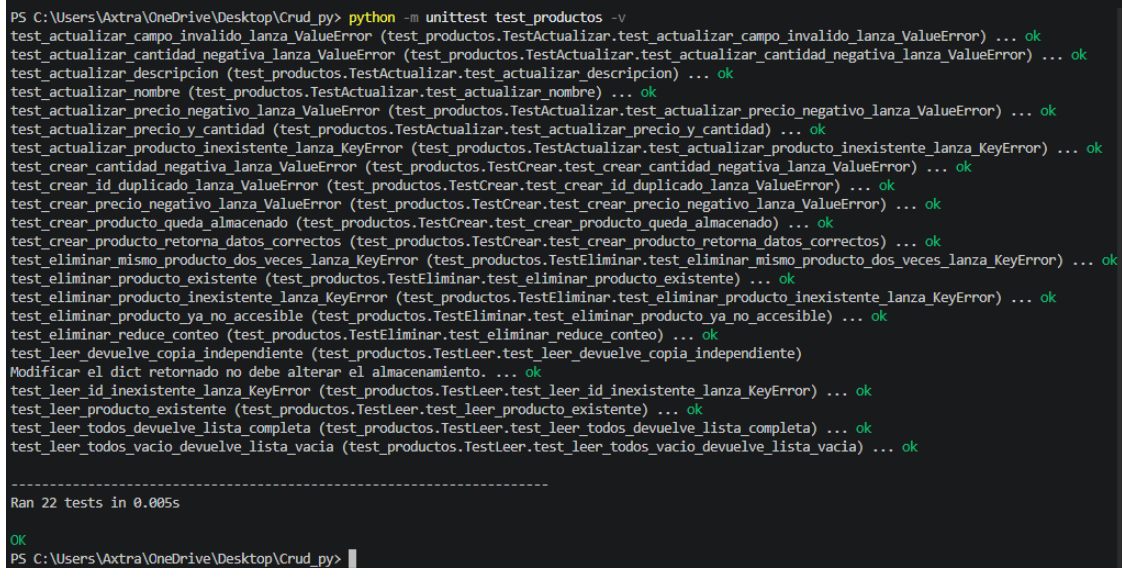
    def test_eliminar_mismo_producto_dos_veces_lanza_KeyError(self):
        self.crud.eliminar(2)
        with self.assertRaises(KeyError): self.crud.eliminar(2)

if __name__ == "__main__":
    unittest.main(verbosity=2)

```

6.2 Resultado de ejecución de los tests

Resultado al ejecutar: `python -m unittest test_productos -v`



```

PS C:\Users\Axtra\OneDrive\Desktop\Crud_py> python -m unittest test_productos -v
test_actualizar_campo_invalido_lanza ValueError (test_productos.TestActualizar.test_actualizar_campo_invalido_lanza ValueError) ... ok
test_actualizar_cantidad_negativa_lanza ValueError (test_productos.TestActualizar.test_actualizar_cantidad_negativa_lanza ValueError) ... ok
test_actualizar_descripcion (test_productos.TestActualizar.test_actualizar_descripcion) ... ok
test_actualizar_nombre (test_productos.TestActualizar.test_actualizar_nombre) ... ok
test_actualizar_precio_negativo_lanza ValueError (test_productos.TestActualizar.test_actualizar_precio_negativo_lanza ValueError) ... ok
test_actualizar_precio_y_cantidad (test_productos.TestActualizar.test_actualizar_precio_y_cantidad) ... ok
test_actualizar_producto_inexistente_lanza KeyError (test_productos.TestActualizar.test_actualizar_producto_inexistente_lanza KeyError) ... ok
test_crear_cantidad_negativa_lanza ValueError (test_productos.TestCrear.test_crear_cantidad_negativa_lanza ValueError) ... ok
test_crear_id_duplicado_lanza ValueError (test_productos.TestCrear.test_crear_id_duplicado_lanza ValueError) ... ok
test_crear_precio_negativo_lanza ValueError (test_productos.TestCrear.test_crear_precio_negativo_lanza ValueError) ... ok
test_crear_producto_queda almacenado (test_productos.TestCrear.test_crear_producto_queda almacenado) ... ok
test_crear_producto_retorna_datos_correctos (test_productos.TestCrear.test_crear_producto_retorna_datos_correctos) ... ok
test_eliminar_mismo_producto_dos_veces_lanza KeyError (test_productos.TestEliminar.test_eliminar_mismo_producto_dos_veces_lanza KeyError) ... ok
test_eliminar_producto_existente (test_productos.TestEliminar.test_eliminar_producto_existente) ... ok
test_eliminar_producto_inexistente_lanza KeyError (test_productos.TestEliminar.test_eliminar_producto_inexistente_lanza KeyError) ... ok
test_eliminar_producto_ya_no_accesible (test_productos.TestEliminar.test_eliminar_producto_ya_no_accesible) ... ok
test_eliminar_reduce conteo (test_productos.TestEliminar.test_eliminar_reduce conteo) ... ok
test_leer_devuelve copia independiente (test_productos.TestLeer.test_leer_devuelve copia independiente)
Modificar el dict retornado no debe alterar el almacenamiento. ... ok
test_leer_id_inexistente_lanza KeyError (test_productos.TestLeer.test_leer_id_inexistente_lanza KeyError) ... ok
test_leer_producto_existente (test_productos.TestLeer.test_leer_producto_existente) ... ok
test_leer_todos_devuelve_lista_completa (test_productos.TestLeer.test_leer_todos_devuelve_lista_completa) ... ok
test_leer_todos_vacio_devuelve_lista_vacia (test_productos.TestLeer.test_leer_todos_vacio_devuelve_lista_vacia) ... ok
-----
Ran 22 tests in 0.005s
OK
PS C:\Users\Axtra\OneDrive\Desktop\Crud_py>

```

Figura: 22 pruebas unitarias — Ran 22 tests in 0.00s OK

7. Prueba de Calidad del Código — SonarCloud

Se realizó el análisis de calidad del código utilizando SonarCloud (versión online de SonarQube), integrado con el repositorio de GitHub. El análisis evalúa fiabilidad, seguridad, mantenibilidad, cobertura y duplicaciones.

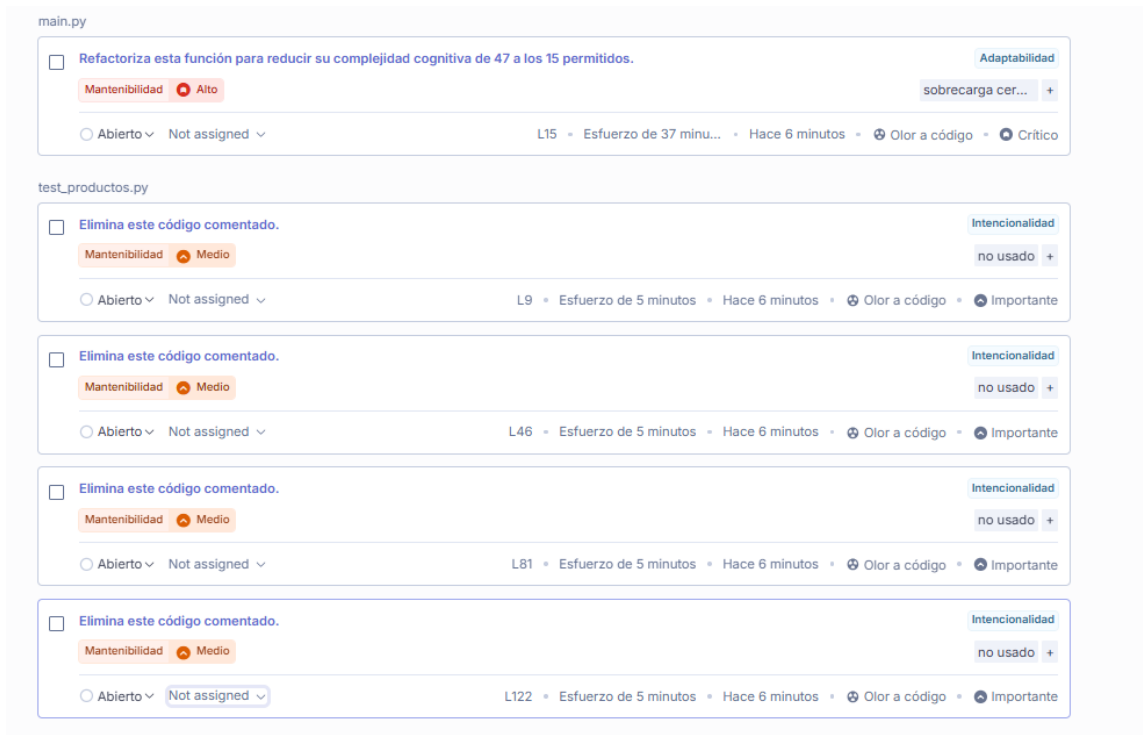


Figura: Dashboard de resultados en SonarCloud

8. Control de Versiones — GitHub

El código fuente fue publicado en un repositorio de GitHub y compartido con el docente al correo jonathansanchez2948@correo.itm.edu.co.

Campo	Valor
URL del repositorio	https://github.com/DurangoJimmy/Crud/tree/main/Crud_py/Crud_py/Crud_py

DurangoJimmy / Crud

Q Type to search

+

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Files

main

+

Q

Q Go to file

T

Crud_py/Crud_py/Crud_py

> Capturas

> __pycache__

Guia4_Entrega_CRUD_Python.d...

generar_documento.py

main.py

productos.json

productos.py

sonar-project.properties

test_productos.py

Crud_py.zip

Crud / Crud_py / Crud_py / Crud_py /

Add file

...

DurangoJimmy Add files via upload

ed20e15 · now

History

Name	Last commit message	Last commit date
..		
Capturas	Add files via upload	now
__pycache__	Add files via upload	now
Guia4_Entrega_CRUD_Python.docx	Add files via upload	now
generar_documento.py	Add files via upload	now
main.py	Add files via upload	now
productos.json	Add files via upload	now
productos.py	Add files via upload	now

9. Bibliografía

- Python Software Foundation. (2024). Python 3 Documentation. <https://docs.python.org/3/>
- Python Software Foundation. (2024). unittest — Unit testing framework. <https://docs.python.org/3/library/unittest.html>
- Python Software Foundation. (2024). json — JSON encoder and decoder. <https://docs.python.org/3/library/json.html>
- Martin, R. C. (2009). Clean Code: A Handbook of Agile Software Craftsmanship. Prentice Hall.
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- SonarSource. (2024). SonarCloud Documentation. <https://docs.sonarcloud.io/>
- GitHub, Inc. (2024). GitHub Docs. <https://docs.github.com/>